SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO4: Implement machine learning techniques including decision tree algorithms, reinforcement learning, and build models for classification and decision-making. (BL3)

Assessment Tool Weightage: 50% **Duration: 1 Hour**

QUESTIONS: 4 **Total Marks:40**

Annexure A

Code of Conduct:

I **Anchal Kumari(192312255) (**Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Anchal

Signature of the student:

Annexure A

INDUSTRY PROBLEM TASK

INDUSTRY SCENARIO

Context: A healthcare company has collected patient records (age, blood pressure, cholesterol, glucose level, BMI, family history) to build a predictive model for early diagnosis of diabetes. The company also wants to train an AI agent to recommend personalised treatment actions (Diet / Exercise / Medication / Monitor) based on patient state transitions over time. You are hired as an AI/ML Engineer. Address the following tasks:

Task 1: Data Preparation & Inductive Learning

(a) Inductive Learning Approach, Target Variable and Key Features

Inductive learning here means learning a general classification rule from a finite set of labelled patient records (examples) and generalising it to unseen patients — i.e., supervised inductive learning. Each record is an instance described by attribute values, and the model induces a hypothesis h: $X \rightarrow Y$ that best fits the training examples while generalising to new patients (standard PAC-style inductive bias: prefer the simplest hypothesis consistent with the data, e.g. Occam's razor, which is why tree-based and linear models are preferred for interpretability in healthcare).

Target variable: Diabetes_Status (binary: 1 = Diabetic, 0 = Non-diabetic).

Three key features and justification:

- Glucose level — directly reflects blood sugar regulation and is the clinical marker most proximally linked to diabetes diagnosis (e.g., fasting glucose ≥126 mg/dL is a diagnostic threshold).
- BMI (Body Mass Index) — obesity is a well-established risk factor for Type-2 diabetes through insulin resistance; higher BMI correlates strongly with disease incidence.
- Family history — captures genetic predisposition; diabetes has a hereditary component, so this feature adds information not present in the current physiological readings.
- (Supporting) Age and Blood Pressure are also relevant since risk rises with age and diabetes frequently co-occurs with hypertension, but glucose, BMI and family history are the strongest independent predictors.

(b) Handling Class Imbalance

Medical datasets of this kind typically have far more non-diabetic than diabetic records. Training on the raw imbalanced data biases the model toward the majority class and increases the clinically dangerous error type — False Negatives (missed diabetic patients).

Recommended strategy: SMOTE (Synthetic Minority Oversampling Technique), combined with class weighting.

- SMOTE generates synthetic minority-class (diabetic) samples by interpolating between existing minority neighbours in feature space, rather than merely duplicating rows — this reduces overfitting compared to naive random oversampling.
- Under-sampling the majority class is avoided as the sole strategy because it discards potentially useful non-diabetic records and shrinks an already limited clinical dataset.
- Class weights (e.g., inversely proportional to class frequency) are applied additionally in the loss function/split-criterion so the learning algorithm penalises misclassifying the minority (diabetic) class more heavily — a low-cost complementary safeguard.

Justification: In healthcare, missing a diabetic patient (False Negative) is far costlier than a false alarm (False Positive), so the balancing strategy is chosen specifically to raise Recall on the diabetic class.

(c) Train-Test Split and Preprocessing

Split ratio: 70:30 (70% train, 30% test), using stratified sampling.

Stratification preserves the diabetic: non-diabetic ratio in both splits. A 70:30 split is preferred over a more aggressive 80:20 or 90:10 split for a medical use-case because the dataset is comparatively small and the minority (diabetic) class is already scarce — a larger held-out test set (30%) gives a more statistically reliable estimate of clinical performance (Recall, F1) before deployment, which matters more than squeezing out a marginal training-accuracy gain.

Preprocessing steps applied:

- Normalisation / Standardisation — numeric features (age, BP, cholesterol, glucose, BMI) are scaled (e.g., Min-Max or Z-score) so that features with larger natural ranges (e.g., cholesterol in mg/dL) do not dominate distance- or gradient-based components of the pipeline; this mainly benefits Logistic Regression / Naïve Bayes-style statistical models, while tree-based models are scale-invariant but still benefit from consistent handling.
- Encoding — categorical fields such as family history (Yes/No) are label- or one-hot encoded so they can be consumed numerically by the models.
- Missing-value imputation — median imputation for skewed clinical numeric fields (e.g., glucose) and mode imputation for categorical fields, to avoid dropping valuable patient rows.
- Impact: normalisation stabilises and speeds convergence for statistical models; encoding makes categorical predictors usable; imputation preserves sample size, which matters given the class imbalance already present.

Task 2: Decision Tree for Diagnosis

(a) Decision Tree — First Three Levels and Root Node Justification

Root node selection: Information Gain (or equivalently, lowest weighted Gini Index after the split) is computed for every candidate feature, and the feature that produces the greatest reduction in impurity is chosen as the root. Based on typical clinical predictive power, Glucose Level yields the highest Information Gain because it separates diabetic from non-diabetic records most cleanly (its threshold split produces the purest child nodes), so it is selected as the root.

Feature	Gini Index (post-split, illustrative)	Information Gain	Selected?
Glucose Level	0.21	0.34	Yes — Root
BMI	0.29	0.26	No — Level 2
Age	0.35	0.18	No
Family History	0.33	0.20	No
Blood Pressure	0.37	0.15	No

(Illustrative values — in practice these are computed from the actual dataset using scikit-learn's DecisionTreeClassifier(criterion='gini' or 'entropy') and .feature_importances_.)

First three levels of the tree:

- Level 1 (Root): Glucose Level ≤ 126 mg/dL ?
  - → Yes branch → Level 2: BMI ≤ 30 ?
    - → Yes → Level 3: Age ≤ 45 ? → leaf leaning Non-Diabetic
    - → No → Level 3: Family History = Yes ? → leaf leaning Borderline/Diabetic
  - → No branch (Glucose > 126) → Level 2: BMI ≤ 25 ?
    - → Yes → Level 3: Blood Pressure ≤ 130 ? → leaf leaning Diabetic (moderate risk)
    - → No → Level 3: Family History = Yes ? → leaf strongly Diabetic (high risk)

(b) Model Evaluation

Metric	Formula	Illustrative Value
Accuracy	(TP+TN) / (TP+TN+FP+FN)	0.86
Precision	TP / (TP+FP)	0.81
Recall (Sensitivity)	TP / (TP+FN)	0.74
F1-Score	2·(Precision·Recall)/(Precision+Recall)	0.77

False Negatives here represent diabetic patients incorrectly predicted as non-diabetic — the most clinically dangerous error, since these patients would not receive timely treatment.

Ways to reduce False Negatives (with clinical justification):

- Lower the classification decision threshold for the 'Diabetic' class so borderline cases are flagged for further testing rather than dismissed — trades some Precision for higher Recall, which is acceptable in screening contexts.
- Apply the SMOTE/class-weighting from Task 1(b) specifically to make the tree's splitting criterion more sensitive to the minority class.

- Use Recall / F1 (not Accuracy) as the primary model-selection metric, since Accuracy can look high while still missing many true diabetics when classes are imbalanced.
- Add a clinical safety net: route all model-flagged 'borderline' cases to a confirmatory lab test (e.g., HbA1c) rather than relying on the model's binary output alone.

(c) Post-Pruning (Cost-Complexity Pruning)

Cost-complexity pruning finds the subtree that minimises $R\alpha(T) = R(T) + \alpha \cdot |T|$, where $R(T)$ is the tree's misclassification cost, $|T|$ is the number of leaf nodes, and α (ccp_alpha) is a complexity penalty. Increasing α progressively prunes weaker branches (those that reduce impurity the least) to control overfitting.

Model Variant	Training Accuracy	Test Accuracy	Notes
Unpruned / Pre-pruned (max_depth capped early)	0.94	0.79	High variance; overfits noise in small clinical dataset
Post-pruned (cost-complexity, optimal α)	0.88	0.86	Lower training accuracy but better, more stable generalisation

Comparison and recommendation: pre-pruning (stopping growth early via max_depth/ min_samples_leaf) is fast but risks under-fitting if the limits are set too conservatively, or overfitting if too permissive, since it decides without seeing the full tree. Post-pruning grows the full tree first and then removes branches that do not improve validation performance, generally yielding better generalisation. For clinical deployment, the post-pruned tree is more appropriate because it is more stable across patient sub-populations, less likely to memorise noise in a limited dataset, and produces a smaller, more interpretable tree that clinicians can audit — interpretability and stability being essential for regulatory and trust reasons in healthcare AI.

Task 3: Statistical Learning for Risk Stratification

(a) Naïve Bayes / Logistic Regression vs Decision Tree

Logistic Regression is applied as the statistical learning model: it estimates P(Diabetic | features) using a sigmoid over a weighted linear combination of the (normalised) features, which also naturally yields a risk score suitable for stratification (low/medium/high risk), not just a binary label.

Model	Accuracy	AUC-ROC
Decision Tree (post-pruned)	0.86	0.88

Model	Accuracy	AUC-ROC
Logistic Regression	0.84	0.90

Discussion — when a statistical model is preferable: Logistic Regression tends to produce smoother, better-calibrated probability estimates (often reflected in a slightly higher AUC-ROC) and is preferable when (i) the relationship between features and outcome is approximately linear/monotonic, (ii) the dataset is small and a low-variance, low-complexity model is wanted to avoid overfitting, (iii) interpretability of coefficients (odds ratios) is needed for clinical justification of risk, and (iv) probability calibration for risk stratification (e.g., assigning patients to Low/Medium/High risk bands) matters more than a hard classification boundary. The Decision Tree is preferable when relationships are non-linear or involve feature interactions (e.g., BMI's effect depends on age) that a linear model cannot capture directly, and when a fully transparent, rule-based explanation ('if glucose > X and BMI > Y') is required for clinicians.

(b) Feature Importance Comparison

Rank	Decision Tree (Gini importance)	Logistic Regression (|standardised coefficient|)
1	Glucose Level	Glucose Level
2	BMI	Family History
3	Family History	BMI

Do they agree? Largely yes — both models agree that Glucose Level is the single strongest predictor, and both identify BMI and Family History as the next most important features, only differing in the relative order of ranks 2 and 3. This agreement across a non-linear (tree-based) and a linear (statistical) model strengthens clinical confidence that these three features are genuinely the primary drivers of diabetes risk in this dataset, rather than an artifact of one algorithm's inductive bias. The minor rank swap (BMI vs Family History) is expected because Logistic Regression captures a feature's independent linear contribution while the Decision Tree can capture BMI's effect jointly/interactively with other splits, slightly changing its measured importance.

Task 4: Reinforcement Learning for Treatment Recommendation

(a) MDP Formulation

MDP Component	Definition for this problem	Justification
States (S)	Patient health stages, e.g. S0 = Critical, S1 = At-Risk, S2 = Stable, S3 = Healthy (derived by discretising glucose/BMI/BP into risk bands)	Captures the patient's evolving health status so the agent can tailor actions to where the patient currently stands
Actions (A)	{Diet, Exercise, Medication, Monitor}	These are exactly the interventions clinicians can prescribe, keeping the action

MDP Component	Definition for this problem	Justification
		space clinically actionable and small enough to learn efficiently
Reward (R)	R = w1·ΔHealthScore − w2·SideEffectRisk − w3·Cost, e.g. +10 for moving to a healthier state, −10 for worsening, small negative step-cost to discourage inaction	Ties the numeric reward to the real clinical objective (health improvement) while penalising unnecessary medication risk/cost, aligning the agent's incentive with patient welfare
Transition Dynamics P(s'|s,a)	Probabilistic — e.g. from At-Risk, Exercise → Stable with prob. 0.6, stays At-Risk with prob. 0.35, worsens to Critical with prob. 0.05	Patient response to treatment is inherently stochastic (biological variability), so transitions must be modelled probabilistically rather than deterministically

(b) Q-Learning Update

Update rule: $Q(s,a) \leftarrow Q(s,a) + \alpha\,[\, r + \gamma \cdot \max_{a'} Q(s',a') - Q(s,a)\,]$, with learning rate $\alpha = 0.1$, discount factor $\gamma = 0.9$. The agent is trained over 5 patient episodes (each episode = one simulated patient's trajectory through states until reaching Healthy or a fixed horizon).

State-Action Pair	Q-value — Iteration 1	Observed (r, s')	Q-value — Iteration 2 (after update)
(At-Risk, Exercise)	0.00	r = +8, s' = Stable (maxQ(Stable,·)=5.0)	0.00 + 0.1[8 + 0.9(5.0) − 0.00] = 1.25
(At-Risk, Medication)	0.00	r = +6, s' = Stable (maxQ(Stable,·)=5.0)	0.00 + 0.1[6 + 0.9(5.0) − 0.00] = 1.05
(Critical, Medication)	0.00	r = +9, s' = At-Risk (maxQ(At-Risk,·)=1.25 after above)	0.00 + 0.1[9 + 0.9(1.25) − 0.00] = 1.01

Convergence criterion used: training stops when the maximum change in any Q-value between successive sweeps over all state-action pairs falls below a small threshold, e.g. $\max|Q_{new}(s,a) - Q_{old}(s,a)| < \varepsilon$ ($\varepsilon = 0.01$), or alternatively after a fixed number of episodes (e.g., 500) once the policy ($\text{argmax}_a\, Q(s,a)$ for every s) stops changing across episodes — whichever is reached first.

(c) Final Learned Policy and Ethical Considerations

State	Learned Optimal Action π*(s)
Critical	Medication
At-Risk	Exercise
Stable	Monitor

State	Learned Optimal Action $\pi^*(s)$
Healthy	Monitor

Extracted policy: $\pi^*(s) = \text{argmax}_a\ Q(s,a)$ — i.e., for each state, the agent recommends the action with the highest learned Q-value, giving a full state → action lookup table (shown above) that a clinician-facing system can present as a recommendation.

Ethical considerations of deploying an RL agent for medical treatment recommendation:

- Patient safety: the agent must never fully replace clinical judgement — recommendations should be presented as decision support with a qualified clinician retaining final authority ('human-in-the-loop'), especially since RL policies learned from limited/simulated data may not generalise to rare or complex patient presentations.
- Bias: if training episodes are derived from historical data skewed toward certain demographics, the learned policy may under-recommend effective treatments for under-represented patient groups; fairness audits across age, gender and other subgroups are needed before deployment.
- Explainability: unlike the Decision Tree, a Q-table/Q-network's recommendation is not inherently interpretable; the deployed system should expose the reasoning (e.g., expected health-score improvement per action) so clinicians can validate rather than blindly trust the suggestion.
- Reward mis-specification risk: because the reward function is a proxy for 'good health outcome', a poorly designed reward could cause the agent to optimise for the wrong objective (e.g., minimising short-term cost at the expense of long-term patient health), so reward design requires clinical domain expert sign-off and ongoing monitoring after deployment.
- Consent and accountability: patients should be informed that an AI system contributed to their treatment recommendation, and clear accountability must be established for adverse outcomes traced to the model's suggestion.